

使用 PostgreSQL 適用的 Cloud SQL，將 JavaScript 應用程式部署至 Cloud Run

來源：<https://codelabs.developers.google.com/codelabs/deploy-application-with-database/cloud-sql-nodejs?hl=zh-tw>

步驟數：10

目錄

1. [總覽](#)
2. [必要條件](#)
3. [專案設定](#)
4. [開啟 Cloud Shell 編輯器](#)
5. [啟用 API](#)
6. [設定服務帳戶](#)
7. [建立 Cloud SQL 資料庫](#)
8. [準備應用程式](#)
9. [將應用程式部署至 Cloud Run](#)
10. [恭喜](#)

步驟 1 · 約 0 分鐘

1. 總覽

[Cloud Run](#) 是全代管無伺服器平台，能讓您執行可透過 HTTP 要求叫用的無狀態容器。本程式碼研究室將說明如何將 Cloud Run 上的 Node.js 應用程式連線至 PostgreSQL 適用的 [Cloud SQL](#) 資料庫。

學習目標

本實驗室的學習內容包括：

- 建立 PostgreSQL 適用的 Cloud SQL 執行個體
- 將應用程式部署至 Cloud Run，並連線至 Cloud SQL 資料庫

學習目標

- 建立 PostgreSQL 適用的 Cloud SQL 執行個體
 - 將應用程式部署至 Cloud Run，並連線至 Cloud SQL 資料庫
-

步驟 2 · 約 0 分鐘

2. 必要條件

1. 如果沒有 Google 帳戶，請先[建立帳戶](#)。
 - 請改用個人帳戶，而非公司或學校帳戶。公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。
-

步驟 3 · 約 0 分鐘

3. 專案設定

1. 登入 [Google Cloud 控制台](#)。
 2. 在 Cloud 控制台中[啟用帳單](#)。
 - 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
 - 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
 - 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。
 3. [建立新專案](#)，或選擇重複使用現有專案。
-

步驟 4 · 約 0 分鐘

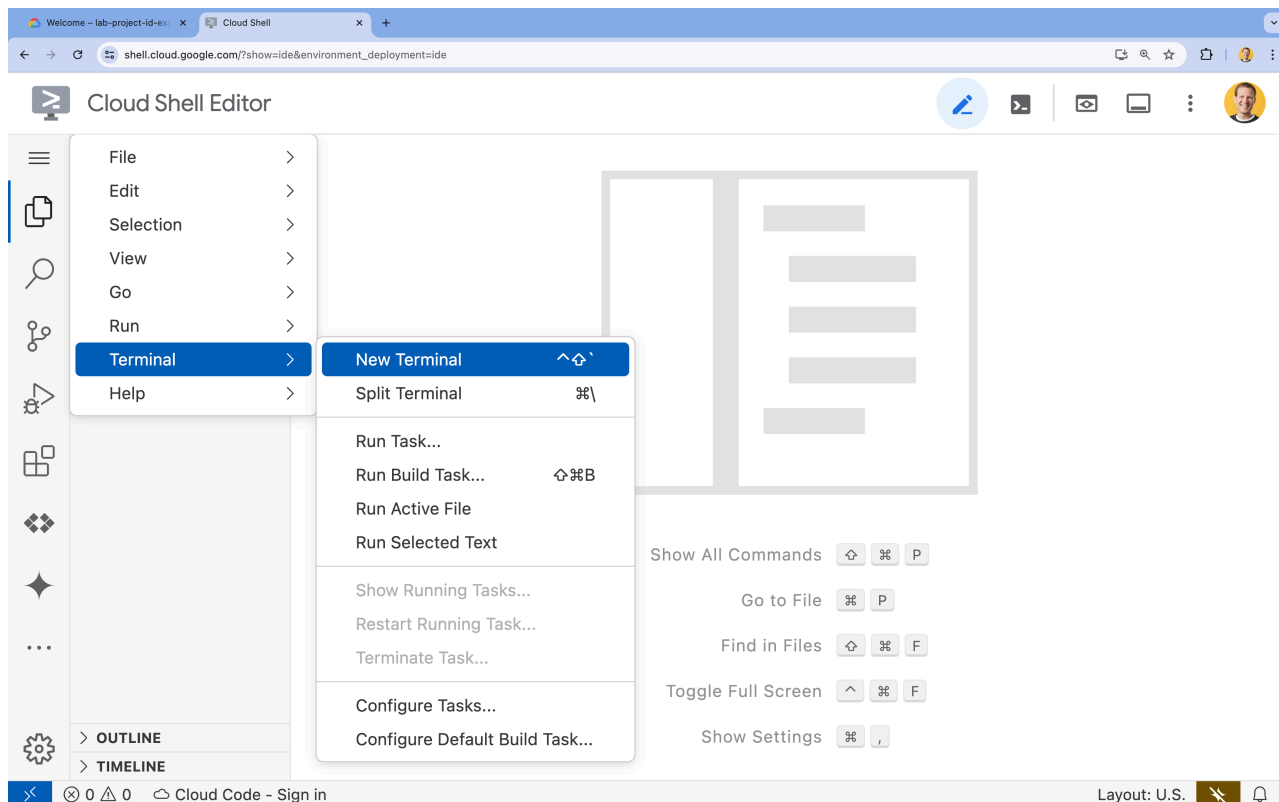
4. 開啟 Cloud Shell 編輯器

1. 前往 [Cloud Shell 編輯器](#)
2. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下漢堡選單



- 按一下「終端機」。
- 按一下「New Terminal」(新增終端機)



3. 在終端機中，使用下列指令設定專案：

- 格式：

```
gcloud config set project [PROJECT_ID]
```

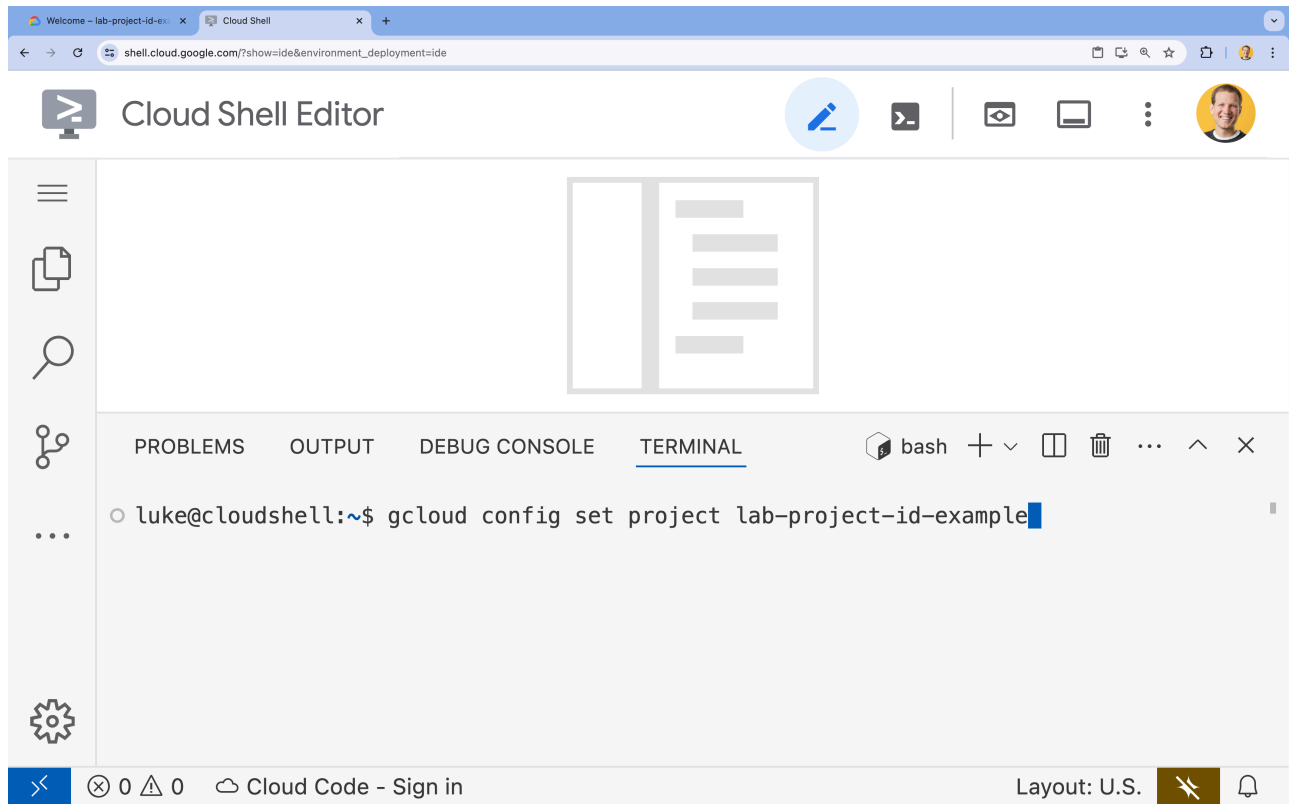
- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，請按照下列步驟操作：

- 您可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```



4. 如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/N)?` 提示，可能是輸入的專案 ID 有誤。按下 `N` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

步驟 5 · 約 0 分鐘

5. 啟用 API

在終端機中啟用 API：

```
gcloud services enable \  
  compute.googleapis.com \  
  sqladmin.googleapis.com \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  networkconnectivity.googleapis.com \  
  servicenetworking.googleapis.com \  
  clouddaicompanion.googleapis.com
```

如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

這個指令可能需要幾分鐘才能完成，但最終應該會產生類似以下的成功訊息：

```
Operation "operations/acf.p2-73d90d00-47ee-447a-b600" finished successfully.
```

6. 設定服務帳戶

建立及設定 Google Cloud 服務帳戶，供 Cloud Run 使用，確保該帳戶具備連線至 Cloud SQL 的正確權限。

1. 執行下列 `gcloud iam service-accounts create` 指令，建立新的服務帳戶：

```
gcloud iam service-accounts create quickstart-service-account \
  --display-name="Quickstart Service Account"
```

2. 執行下列 `gcloud projects add-iam-policy-binding` 指令，將「記錄檔寫入者」角色新增至您剛建立的 Google Cloud 服務帳戶。

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
  --member="serviceAccount:quickstart-service-
account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --role="roles/logging.logWriter"
```

7. 建立 Cloud SQL 資料庫

1. 建立服務連線政策，允許 Cloud Run 透過 [Private Service Connect](#) 連線至 Cloud SQL

```
gcloud network-connectivity service-connection-policies create quickstart-policy \
  --network=default \
  --project=${GOOGLE_CLOUD_PROJECT} \
  --region=us-central1 \
  --service-class=google-cloud-sql \
  --
subnets=https://www.googleapis.com/compute/v1/projects/${GOOGLE_CLOUD_PROJECT}/regions/us
-central1/subnetworks/default
```

2. 為資料庫產生專屬密碼

```
export DB_PASSWORD=$(openssl rand -base64 20)
```

3. 執行 `gcloud sql instances create` 指令，建立 Cloud SQL 執行個體

```
gcloud sql instances create quickstart-instance \
  --project=${GOOGLE_CLOUD_PROJECT} \
  --root-password=${DB_PASSWORD} \
  --database-version=POSTGRES_17 \
  --tier=db-perf-optimized-N-2 \
  --region=us-central1 \
  --ssl-mode=ENCRYPTED_ONLY \
  --no-assign-ip \
  --enable-private-service-connect \
  --psc-auto-
connections=network=projects/${GOOGLE_CLOUD_PROJECT}/global/networks/default
```

這個指令可能需要幾分鐘才能完成。

1. 執行 `gcloud sql databases create` 指令，在 `quickstart-instance` 中建立 Cloud SQL 資料庫。

```
gcloud sql databases create quickstart_db \
  --instance=quickstart-instance
```

步驟 8 · 約 0 分鐘

8. 準備應用程式

準備回應 HTTP 要求的 Node.js 應用程式。

1. 在 Cloud Shell 建立名為 `helloworld` 的新目錄，然後前往該目錄：

```
mkdir helloworld  
cd helloworld
```

2. 將 `package.json` 檔案初始化為模組。

```
npm init -y  
npm pkg set type="module"  
npm pkg set main="index.mjs"  
npm pkg set scripts.start="node index.mjs"
```

3. 安裝 `pg`，與 PostgreSQL 資料庫互動。

```
npm install pg
```

4. 安裝 Express，接受傳入的 HTTP 要求。

```
npm install express
```

5. 建立包含應用程式程式碼的 `index.mjs` 檔案。這組代碼可執行下列操作：

- 接受 HTTP 要求
- 連線至資料庫
- 將 HTTP 要求的時間儲存在資料庫中
- 傳回最近五次要求的時間

在 Cloud Shell 中執行下列指令：

```

cat > index.mjs << "EOF"
import express from 'express';
import pg from 'pg';
const { Pool } = pg;

const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  ssl: {
    require: true,
    rejectUnauthorized: false, // required for self-signed certs
    // https://node-postgres.com/features/ssl#self-signed-cert
  }
});

const app = express();

app.get('/', async (req, res) => {
  await pool.query('INSERT INTO visits(created_at) VALUES(NOW())');
  const {rows} = await pool.query('SELECT created_at FROM visits ORDER BY created_at DESC
LIMIT 5');
  console.table(rows); // prints the last 5 visits
  res.send(rows);
});

const port = parseInt(process.env.PORT) || 8080;
app.listen(port, async () => {
  console.log('process.env: ', process.env);
  await pool.query(`CREATE TABLE IF NOT EXISTS visits (
    id SERIAL NOT NULL,
    created_at timestamp NOT NULL,
    PRIMARY KEY (id)
  );`);
  console.log(`helloworld: listening on port ${port}`);
});

EOF

```

這段程式碼會建立基本的網路伺服器，用於監聽 PORT 環境變數定義的通訊埠。應用程式現在已可部署。

步驟 9 · 約 0 分鐘

9. 將應用程式部署至 Cloud Run

1. 執行下列指令，將應用程式部署至 Cloud Run：

```
gcloud run deploy helloworld \
  --region=us-central1 \
  --source=. \
  --set-env-vars DB_NAME="quickstart_db" \
  --set-env-vars DB_USER="postgres" \
  --set-env-vars DB_PASSWORD=${DB_PASSWORD} \
  --set-env-vars DB_HOST="$(gcloud sql instances describe quickstart-instance --
project=${GOOGLE_CLOUD_PROJECT} --
format='value(settings.ipConfiguration.pscConfig.pscAutoConnections.ipAddress)')" \
  --service-account="quickstart-service-
account@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --network=default \
  --subnet=default \
  --allow-unauthenticated
```

2. 如果系統提示您確認是否要繼續，請按下 **Y** 和 **Enter**：

```
Do you want to continue (Y/n)? Y
```

幾分鐘後，應用程式應會提供網址供您造訪。

前往該網址，即可查看應用程式的運作情形。每次造訪網址或重新整理頁面時，系統都會以 JSON 格式傳回最近五次造訪記錄。

10. 恭喜

在本實驗室中，您已學會如何執行下列工作：

- 建立 PostgreSQL 適用的 Cloud SQL 執行個體
- 將應用程式部署至 Cloud Run，並連線至 Cloud SQL 資料庫

清除所用資源

Cloud SQL 沒有免費方案，如果您繼續使用，系統會向您收費。您可以刪除 Cloud 專案，以免產生額外費用。

不使用服務時，Cloud Run 不會收費，但您可能仍須支付容器映像檔在 Artifact Registry 的儲存費用。刪除 Cloud 專案後，系統就會停止對專案使用的所有資源收取費用。

如要刪除專案，請按照下列步驟操作：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

您也可以從 Cloud Shell 磁碟刪除不必要的資源。您可以：

1. 刪除 Codelab 專案目錄：

```
rm -rf ~/task-app
```

2. 警告！這項操作無法復原，如要刪除 Cloud Shell 中的所有內容來釋出空間，可以[刪除整個主目錄](#)。請務必將要保留的內容另存到其他位置。

```
sudo rm -rf $HOME
```

持續掌握情況

- 使用 [Cloud SQL Node.js 連接器](#)，將全端 Next.js 應用程式部署至 Cloud Run，並搭配 PostgreSQL 適用的 Cloud SQL
 - 使用 [Cloud SQL Node.js 連接器](#)，將全端 Angular 應用程式部署至 Cloud Run，並搭配 PostgreSQL 適用的 Cloud SQL
 - 使用 [Node.js Admin SDK](#)，將全端 Angular 應用程式部署至 Cloud Run，並搭配使用 Firestore
 - 使用 [Node.js Admin SDK](#)，將全端 Next.js 應用程式部署至 Cloud Run，並搭配 Firestore 使用
-